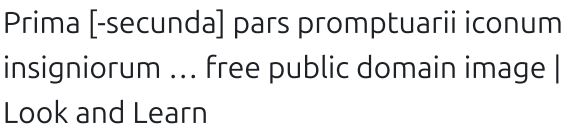
Pràctica 1

En els darrers anys han aparegut nombroses plataformes per construir aplicacions basades en intel·ligència artificial. Aquestes plataformes ja no només permeten escollir un model de llenguatge, sinó que també gestionen **prompts reutilitzables** i **eines (tools)** que poden ser invocades pels models o per agents autònoms per resoldre tasques concretes.



Prima [-secunda] pars promptuarii iconum insigniorum … free public domain image | Look and Learn

És habitual que els desenvolupadors mantinguin una biblioteca compartida de prompts i eines Python (i en altres llenguatges) que posteriorment són utilitzades tant per usuaris humans com per aplicacions automatitzades. Per aquest motiu, és necessari disposar d'un servidor que permeti registrar, consultar i administrar aquests recursos mitjançant una API REST.

En aquesta primera pràctica implementarem el **back-end RESTful** d'aquest sistema. En una segona pràctica es desenvoluparà un client web amb Jakarta MVC que consumirà aquesta API. Finalment, de forma completament opcional, es podrà integrar un client autònom executat dins del navegador mitjançant **WebAssembly** capaç de consumir la mateixa API REST.

1. Objectius

L'objectiu principal és construir una API Web que permeti gestionar tres recursos principals:

?

- usuaris,
- prompts,
- eines.

Aquests recursos han de poder ser consumits indistintament per un usuari humà mitjançant una aplicació web o per un client automatitzat.

2. Funcionament general

En aquesta primera pràctica només s'haurà d'implementar **el servidor REST** i **el model de dades corresponent**.

L'aplicació actuarà com un repositori centralitzat de **prompts** i **eines**. Cada **prompt** representa una plantilla reutilitzable que descriu una tasca que pot executar un model de llenguatge (LLM). Cada **eina** representa un petit programa reutilitzable que podrà ser executat posteriorment per un client extern.

En aquesta pràctica **no és necessari executar cap prompt ni cap eina**. El servidor únicament serà responsable de gestionar aquests recursos.

Prompt

Cada prompt haurà d'incloure com a mínim:

- identificador
- títol
- descripció breu
- plantilla (*template*)
- autor
- etiquetes
- data de creació

Eina

Cada eina haurà d'incloure

- identificador
- nom
- descripció
- llenguatge
- codi
- autor
- versió
- data de creació

Opcionalment es poden afegir metadades com:

- llibreries requerides
- versió mínima de Python, etc.

?

- categoria

Tota aquesta informació s'haurà de representar correctament mitjançant **JPA**. En concret, s'hauran de crear com a mínim **tres entitats**: Prompt, Tool i Customer.

Els usuaris registrats podran crear i mantenir prompts i eines. **Cal destacar que els prompts poden ser totalment públics o només disponibles pels usuaris registrats, és a dir, privats.**

Si els prompts són accessibles pels usuaris registrats, s'indicaran com a **privats** amb l'atribut corresponent a l'entitat.

3. Enunciat

Haurem de suportar les funcionalitats següents:

1. Definirem una API REST comuna per tothom i que els grups hauran d'implementar a les seves pràctiques.
2. Cal implementar els serveis REST perquè treballin i retornin JSON. Eines que us poden ser d'interès: Gson, JAXB (exemple d'ús)
3. Fer servir versionat per la sostenibilitat de l'API.
4. Cal implementar un client REST per a poder interaccionar i testejar la vostra pràctica. No esteu obligats a fer servir cap tecnologia per implementar els serveis o clients REST, és més, l'essència d'aquest sistema és la simplicitat, i això serà el que més es valorarà de la vostra solució. Suggeriment PostMan API: https://www.postman.com/. Podeu fer servir TDD de manera simple amb Postam també:







3.1 API

Els serveis web exposats relatius als tres recursos principals: **prompts**, **eines** i **usuaris.** Els tres recursos principals hauran de seguir la següent especificació:

Pels prompts:

1. **GET /rest/api/v1/prompt?author=${author}&tag=${tag}**
   Llista tots els prompts disponibles, ordenats per títol. Es poden afegir els següents filtres opcionals:

   - `author=${author}` per mostrar només els prompts creats per un determinat usuari.
   - `tag=${tag}` per mostrar només els prompts etiquetats amb una determinada categoria.

   El filtratge s'ha de realitzar mitjançant una consulta a la base de dades. No s'acceptarà com a vàlid retornar tots els prompts i filtrar-los programàticament amb Java.

   Si el prompt és **pública**, es retornarà la **informació completa**. Si és **privat**, només **l'identificador**, el **títol**, l'**autor** i la **data de creació**.

2. **GET /rest/api/v1/prompt/{id}**

   Retorna tota la informació corresponent al prompt identificat per `{id}`.

   Si el prompt no existeix, cal retornar el codi **HTTP corresponent**.

   Si el prompt és **privat**, només es podrà retornar si l'usuari està registrat (autentificat).

3. **POST /rest/api/v1/prompt**

   Registra un nou prompt a partir d'un document JSON proporcionat pel client.

   S'haurà de validar com a mínim:

   - que el títol no sigui buit;
   - que el contingut del prompt existeixi;
   - que l'autor existeixi al sistema;
   - que les eines associades al prompt existeixin.

   En cas correcte, es retornarà **HTTP 201 Created** amb l'identificador del nou prompt.

   Per aquesta operació cal que l'usuari estigui autentificat.

4. **PUT /rest/api/v1/prompt/{id}**

   Modifica la informació d'un prompt existent. **Opcional**.

   Per aquesta operació, cal que l'usuari estigui autentificat.

5. **GET /rest/api/v1/prompt/{id}/tools**

   Retorna la llista d'**eines** associades al prompt identificat per `{id}`.

   Si el prompt no existeix, o no hi ha eines associades cal retornar el codi **HTTP corresponen**?

   Si el prompt és **privat**, només es podrà retornar si l'usuari està registrat (autentificat).

6. **PUT /rest/api/v1/prompt/{id}/tools**

   Actualitza les **eines** associades al prompt identificat per `{id}`. El client enviarà una llista amb els **identificadors** de les eines que vulgui associar al prompt.

   Per aquesta operació cal que l'usuari estigui autentificat.

Per les eines:

1. **GET /rest/api/v1/tool?author=${author}&language=${language}**
   Llista totes les eines disponibles, ordenades pel seu nom.

   Es poden especificar els següents filtres opcionals:

   - `author=${author}` per mostrar només les eines creades per un determinat usuari.
   - `language=${language}` per mostrar només eines implementades en un determinat llenguatge.

   En aquesta pràctica, totes les eines hauran d'estar implementades en **Python**, però el disseny de l'API ha de permetre incorporar altres llenguatges en un futur.

   El filtratge s'ha de realitzar mitjançant una consulta a la base de dades.

   Si l'eina és **pública**, es retornarà la **informació completa**. Si és **privada**, només **l'identificador**, el **nom**, l'**autor** i el **llenguatge**.

2. **GET /rest/api/v1/tool/{id}**

   Retorna tota la informació corresponent a l'eina identificada per `{id}`.

   Si l'eina no existeix, cal retornar el codi **HTTP corresponent**.

   Si l'eina és privada, només es podrà retornar si l'usuari està registrat (autentificat).

3. **POST /rest/api/v1/tool**

   Registra una nova eina al sistema.

   S'haurà de validar com a mínim:

   - que el nom no sigui buit;
   - que el codi Python existeixi;
   - que l'autor existeixi.

   En cas correcte, es retornarà **HTTP 201 Created** amb l'identificador de la nova eina.

   Per aquesta operació cal que l'usuari estigui autentificat.

4. **PUT /rest/api/v1/tool/{id}**

   Modifica la informació de l'eina identificada per `{id}`. **Opcional**.

?

Per aquesta operació cal que l'usuari estigui autentificat.

5. **DELETE /rest/api/v1/tool/{id}**

   Esborra l'eina del repositori identificada per `{id}`. **Opcional**.

   Per aquesta operació cal que l'usuari estigui autentificat.

Pels usuaris:

1. **GET /rest/api/v1/customer**
   - Llistat JSON dels usuaris.
   - Aquesta crida no pot retornar informació confidencial, p. ex., la contrasenya dels usuaris.
2. **GET /rest/api/v1/customer/{id}**
   - Retorna la informació de l'usuari amb identificador `{id}`.
     - Cal indicar l'**enllaç a l'últim prompt i/o eina que va afegir** per suportar el principi de **HATEOAS**. Per exemple, en JSON:

```
{
  "_links": {
    "last-prompt-added": { "href": "/prompt/12345" },
    "last-tool-added": { "href": "/tool/12345" }
  }
}
```

   - Aquesta crida no pot retornar informació confidencial, p. ex., la contrasenya d'aquest usuari.
3. **PUT /rest/api/v1/customer/{id}**
   - **Opcional!** Modifica les dades del client amb identificador `{id}` al sistema amb les dades JSON/XML proporcionades.
   - Per aquesta operació, cal que el client estigui autentificat.

3.2 Autentificació dels usuaris

Per aquells serveis web que requereixen que el client estigui autentificat, s'ha creat una anotació de tipus **Runtime** Java anomenada **@Secured.** Quan anoteu un mètode amb aquesta anotació, el servidor intentarà autentificar a l'usuari amb el mètode d'autenticació d'accés bàsica (en anglès, **HTTP Basic Authentication**), que usa la capçalera HTTP Authorization per passar al servidor les credencials de l'usuari.

Hi ha formes més professionals de realitzar aquesta autentificació i autorització d'operacions REST via JAX-RS. Com a solució simple i sense seguir cap estàndard, podríeu implementar un petit mec? ne d'API Key Authentication. O fer servir altres mecanismes d'autentificació com JWT, etc.

Resumint tot això, tindrem les següents funcionalitats obligatòries a implementar:

De l'API REST:

1. Les crides API REST obligatòries.
2. Versionat en l'API REST.
3. Dades enviades i rebudes en format JSON.
4. L'autentificació de les crides que així ho requereixen.
5. **Ús de codis HTTP adequats als resultats de les operacions de l'API REST.**
6. Proveir un client per l'API REST.
7. Proveir un joc de proves per aquest client.

Nota. Cal que diferencieu els recursos de l'API REST (**prompts**, **eines** i **customer**) de les entitats o objectes de domini que necessiteu per poder implementar l'API web.

Es permet l'ús d'eines d'IA generativa, com ara els grans models de llenguatge (LLMs), per ajudar en la generació de codi. **Tanmateix, els estudiants hauran d'assumir la responsabilitat total del contingut generat per aquestes eines, amb el risc que pugui ser considerat plagi si es generen pràctiques amb contingut idèntic entre grups.**

Això també inclou la capacitat d'explicar amb detall el contingut generat amb eines d'IA com si hagués estat desenvolupat pel mateix estudiant. **Si les respostes no són satisfactòries, la pràctica es considerarà suspesa i serà qualificada amb un 0**.

Ampliacions opcionals:

Sempre i quant la part obligatòria estigui ben dissenyada i desenvolupada, amb els següents punts s'aconseguirà pujar nota:

Àmbit actuació REST:

- Permetre que els serveis REST rebin i retornin XML com a l'alternativa a JSON.
- Cuinar codis i missatges HTTP per la gestió de les excepcions remotes (ex,. 404 Not Found, 403 Forbidden, 201 Created, ...).
- Ampliar l'API REST per incorporar les parts opcionals indicades.
- Preparació de **tests unitaris** automàtics amb Postman (https://learning.postman.com/docs/writing-scripts/test-scripts/) o altre programari.
- Sistema d'autentificació diferents a l'HTTP Basic Authentication.

4. Documentació

Juntament amb el codi font caldrà lliurar un informe en format **PDF** amb la següent estructura:

- **Introducció**. Presentació general de la pràctica i dels objectius.
- **Estructura de la pràctica**. Per exemple, caldrà especificar l'estructura del projecte, dels fitxers, les entitats JPA, entre d'altres.
- **Decisions de disseny**. Quines alternatives heu considerat a l'hora de fer el projecte i perquè heu

?

escollit l'alternativa escollida enfront de les altres. Aquí, entre altres coses, hauríeu d'explicar quin model dels vistos a classe heu fet servir, quines parts opcionals o millores addicionals heu fet i com heu decidit implementar-les. Heu emprat eines i recursos externs?

- **Jocs de proves realitzats**. Quines proves heu fet per assegurar-vos que el vostre projecte funciona correctament en tots els casos possibles. És molt important realitzar un joc de proves extensiu, que cobreixi tots els casos, tant els positius com els negatius, com aquells que puguin produir excepcions d'algun tipus. Per tot això, heu aplicat TDD i/o BDD?

- **Conclusions**. A quines conclusions heu arribat en finalitzar aquest primer projecte.

- **Manual d'instal·lació**. La instal·lació s'ha de reduir als següents passos:
  1. Obrir projecte NetBeans;
  2. Donar-li a deploy de la vostra pràctica;
  3. (Opcionalment) Executar scripts per crear la base de dades;
  4. Passos per executar el client REST i petit joc de proves.

  A més, ha d'haver-hi un usuari de prova anomenat **sob** i contrasenya **sob** per poder provar el correcte funcionament del mecanisme de les revisions o reviews. Si necessiteu forçosament alguna altra acció, expliciteu-la adequadament.

  La **nom de la vostra base de dades ha de ser** com segueix: "**sob_grup_NN**", on "**NN**" és el número del vostre grup de pràctiques del Moodle.

5. Lliurament del projecte

El projecte es lliurarà via Moodle abans de la data màxima d'entrega.

Cal que entregueu:

1. Un arxiu **.zip** anomenat SOB_P1_nom1_nom2.zip, on nom1 i nom2 seran el nom (i cognoms) de cada un dels components del grup que entrega la pràctica.
   - Hi ha d'haver una carpeta anomenada **projecte** on hi haurà tots els fitxers que formen el projecte en NetBeans, de manera que es pugui obrir directament en el mateix entorn del laboratori.
   - Cal deixar ben clar l'ús dels passos addicionals que siguin necessaris per posar en funcionament la vostra pràctica.
2. Un fitxer anomenat **documentacio.pdf** amb la documentació de la pràctica que segueixi el format explicat en el punt anterior.

6. Avaluació

Cal fer com a mínim les parts obligatòries que us permetran aconseguir fins a un 9.5 de la nota del projecte.

Si implementeu 1 ampliació opcional correctament, la nota que es podrà aconseguir serà fins a ? 10 per aquesta pràctica. És preferible tenir 1 ampliació ben desenvolupada que una quantitat superior i

que estiguin incorrectes.

Alguns dels aspectes que es valoraran són:

- Funcionalitat correcta del projecte
- Estructuració de la pràctica
- Decisions de disseny preses
- Elegància de la solució proposada a nivell de programació
- Seguretat i extensibilitat de l'aplicatiu
- Compliment dels estàndards i convencions a l'hora de desenvolupar una aplicació web en Java+Netbeans (Java Code Conventions).
- Qualitat de la documentació presentada.

Darrera modificació: dijous, 6 d'agost 2026, 16:11

◄ Lliurament segona convocatòria

Salta a l'activitat

Codi base per la pràctica 1 (JDK 17; GlassFish 6) ►

?